Retrieval-Augmented Generation (RAG)

The Complete Guide — concepts, components, every major RAG type, internals, code, evaluation, production and interview Q&A

Inside: Foundations • Architecture • 11 core components • 18+ RAG types (Naive, Advanced, Modular, Hybrid, Multi-Query/Fusion, HyDE, Parent-Document, Contextual, RAPTOR, Conversational, Corrective, Self-RAG, Adaptive, Multi-hop, Agentic, Graph, SQL, Multimodal) • Python code • Evaluation • Failure modes • 50+ interview questions • Roadmap

Prepared for Mansavi Aggarwal

Contents

1. What is RAG and Why Do We Need It? 4

1.1 Problems with a plain LLM that RAG solves . . . 4

1.2 A brief history (useful for interviews) . . . 4

1.3 RAG vs Fine-tuning vs Long Context vs Prompting . . . 4

2. RAG Architecture: The Big Picture 6

2.1 Offline: Indexing (Ingestion) pipeline . . . 6

2.2 Online: Query (Retrieval + Generation) pipeline . . . 6

2.3 What actually happens, step by step (worked example) . . . 6

3. Components of RAG — In Depth 7

3.1 Document Loaders & Parsing . . . 7

3.2 Chunking (Text Splitting) . . . 7

3.3 Embeddings . . . 8

3.4 Vector Database & Indexing . . . 8

3.5 Retriever . . . 9

3.6 Query Transformation (pre-retrieval) . . . 9

3.7 Re-ranking (post-retrieval) . . . 10

3.8 Context Construction & Compression . . . 10

3.9 Prompt Augmentation . . . 11

3.10 Generator (LLM) . . . 11

3.11 Supporting components . . . 11

4. Types of RAG — Explained One by One 12

4.1 Naive (Simple / Vanilla) RAG . . . 12

4.2 Advanced RAG . . . 12

4.3 Modular RAG . . . 13

4.4 Hybrid RAG (Hybrid Search) . . . 13

4.5 Multi-Query RAG & RAG-Fusion . . . 14

4.6 HyDE (Hypothetical Document Embeddings) . . . 14

4.7 Parent-Document, Small-to-Big & Sentence-Window RAG . . . 15

4.8 Contextual Retrieval (Contextual RAG) . . . 15

4.9 RAPTOR (Hierarchical / Tree-Summarisation RAG) . . . 16

4.10 Conversational RAG (RAG with Memory) . . . 16

4.11 Corrective RAG (CRAG) . . . 17

4.12 Self-RAG (Self-Reflective RAG) . . . 17

4.13 Adaptive RAG . . . 18

4.14 Iterative / Multi-hop RAG . . . 18

4.15 Agentic RAG . . . 19

4.16 Graph RAG (Knowledge-Graph RAG) . . . 19

4.17 Structured / SQL RAG (Text-to-SQL) . . . 20
4.18 Multimodal RAG . . . 20
4.19 Other names you may hear . . . 21
4.20 Summary comparison of RAG types . . . 21
5. Hands-on: Building RAG in Python 23
5.1 Naive RAG from scratch (no framework) . . . 23
5.2 Hybrid retrieval (BM25 + dense) with RRF . . . 23
5.3 Re-ranking with a cross-encoder . . . 24
5.4 The same pipeline with LangChain (LCEL) . . . 24
6. Evaluating RAG 25
6.1 Retrieval metrics (need labelled relevant chunks per query) . . . 25
6.2 Generation / end-to-end metrics (RAG Triad and RAGAS) . . . 25
6.3 How to evaluate in practice . . . 25
7. Failure Modes, Fixes & Production Concerns 27
7.1 Common failure points and fixes . . . 27
7.2 Production checklist . . . 27
8. Interview Questions & Answers 28
8.1 Basics . . . 28
8.2 Chunking & Embeddings . . . 28
8.3 Retrieval & Vector DBs . . . 29
8.4 Advanced RAG types . . . 30
8.5 Evaluation . . . 31
8.6 Production & Scenarios . . . 32
8.7 Quick one-liners for rapid-fire rounds . . . 33
9. Learning Roadmap & Project Ideas 34
9.1 Prerequisites . . . 34
9.2 Phased plan . . . 34
9.3 Portfolio project ideas . . . 34
9.4 One-page cheat sheet . . . 34

1. What is RAG and Why Do We Need It?

Retrieval-Augmented Generation (RAG) is a technique where, before an LLM answers a question, the system first **retrieves** relevant information from an external knowledge source (documents, databases, APIs) and then **augments** the prompt with that information so the LLM can **generate** a grounded answer.

Think of it as an **open-book exam** for the LLM. Without RAG, the model answers from memory (its training weights). With RAG, the model is handed the relevant pages of the book first, then answers.

> **Key idea:** RAG = Retrieve (find relevant context) + Augment (put it into the prompt) + Generate (LLM writes the answer using that context).

1.1 Problems with a plain LLM that RAG solves

LLM limitation	What it means	How RAG fixes it
Knowledge cutoff	Model only knows data up to its training date.	Retrieve fresh documents at query time; just re-index new data.
Hallucination	Model confidently invents facts when it doesn't know.	Answer is grounded in retrieved text; prompt says 'answer only from context'.
No private data	Model never saw your company PDFs, tickets, policies.	Index private data in your own vector store; no retraining needed.
No citations	You can't verify where an answer came from.	Each retrieved chunk carries source metadata, so you can cite it.
Costly to update	Fine-tuning for every data change is slow and expensive.	Updating knowledge = adding/removing documents in the index.
Context window limits	You cannot paste 10,000 documents into one prompt.	Retrieve only the top few most relevant chunks.

1.2 A brief history (useful for interviews)

- **2020 – the original RAG paper** (Lewis et al., Facebook AI): combined a dense retriever (DPR) with a seq2seq generator (BART) and trained them together. The term 'RAG' comes from this paper.
- **REALM (Google, 2020)**: retrieval during pre-training. **FiD – Fusion-in-Decoder**: encodes each passage separately and fuses in the decoder.
- **2023 onward**: with ChatGPT/GPT-4, Gemini, Llama, RAG became the default pattern for enterprise LLM apps, usually *without* training the retriever: an off-the-shelf embedding model + vector DB + LLM.
- **Later evolution**: Advanced RAG → Modular RAG → Self-RAG, Corrective RAG, GraphRAG, Agentic RAG, Multimodal RAG.

1.3 RAG vs Fine-tuning vs Long Context vs Prompting

Approach	What changes	Best for	Weakness
Prompt engineering	Only the instructions	Format, tone, simple tasks	No new knowledge
RAG	The context given at runtime	Fresh/private/large factual knowledge, citations	Depends on retrieval quality; extra latency

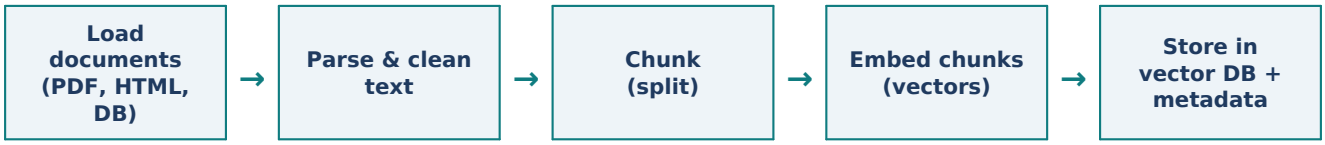
Approach	What changes	Best for	Weakness
Fine-tuning	Model weights	Style, domain language, output format, specific behaviour/skills	Expensive, knowledge goes stale, hard to cite, can forget
Long-context LLM	Paste everything into prompt	Small corpora (fits in window), one-off analysis	Cost per call, 'lost in the middle', doesn't scale to millions of docs
CAG (Cache-Augmented Gen.)	Pre-load whole small corpus into KV cache	Small, static knowledge base, very low latency	Limited by context size; no dynamic retrieval

Interview tip: 'Fine-tuning teaches the model *how* to behave; RAG gives the model *what* to know.' In real systems they are often combined: a fine-tuned model that is also fed retrieved context.

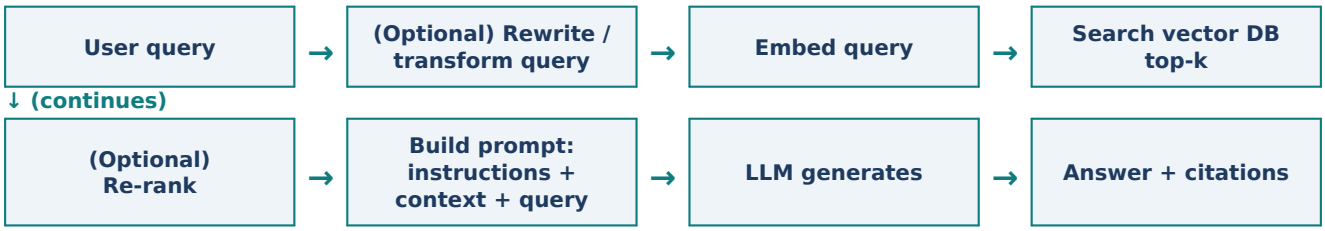
2. RAG Architecture: The Big Picture

Every RAG system has two pipelines: an **offline indexing pipeline** (run once / periodically to prepare the knowledge) and an **online query pipeline** (run on every user question).

2.1 Offline: Indexing (Ingestion) pipeline



2.2 Online: Query (Retrieval + Generation) pipeline



2.3 What actually happens, step by step (worked example)

Suppose an HR bot has the company leave-policy PDF indexed, and the user asks: "*How many sick leaves do I get?*"

1. **Indexing (done earlier):** the 40-page policy PDF was split into ~120 chunks of ~500 tokens. Each chunk was turned into a 768-dimensional vector by an embedding model and stored with metadata {source: 'leave_policy.pdf', page: 7}.
2. **Query embedding:** the question is converted into a vector using the *same* embedding model.
3. **Similarity search:** the vector DB compares the query vector with all chunk vectors (via an ANN index such as HNSW) and returns the top-k (e.g. 4) closest chunks by cosine similarity.
4. **Augmentation:** the chunks are inserted into a prompt template: 'Answer only using the context below. If not found, say you don't know. Context: ... Question: ...'.
5. **Generation:** the LLM reads the context and writes: 'You are entitled to 12 sick leaves per year (Leave Policy, p.7).'

> **Watch out:** Retrieval quality caps answer quality. If the right chunk is not retrieved, even the best LLM cannot answer correctly — 'garbage in, garbage out'. Most RAG debugging is retrieval debugging.

3. Components of RAG — In Depth

Below is every building block of a RAG system, what it does, what happens inside it, and the key design choices.

3.1 Document Loaders & Parsing

Loaders read raw data from sources: PDFs, Word, HTML/web pages, Markdown, Confluence/SharePoint, Notion, databases, CSVs, emails, APIs. Parsing converts them into clean text + metadata.

- **Inside:** text extraction (pypdf, PyMuPDF, pdfplumber), OCR for scanned files (Tesseract), layout-aware parsing for tables/headers (Unstructured, LlamaParse, Docling, Azure Document Intelligence).
- **Metadata** captured here is gold: source, title, page, section, date, author, department, access permissions.
- **Cleaning:** remove headers/footers, page numbers, boilerplate, duplicate text, broken hyphenation; normalise whitespace and encoding.
- **Tables** are a common pain point: convert to Markdown/HTML or create a textual summary per table.

3.2 Chunking (Text Splitting)

Documents are too long to embed and retrieve as a whole, so we split them into smaller **chunks**. A chunk is the unit that gets embedded, retrieved and shown to the LLM.

Why chunk? (1) Embedding models have input limits (e.g. 512 tokens). (2) One vector for a 50-page document averages out all meaning and becomes useless for matching. (3) Smaller relevant chunks = less noise and fewer tokens in the prompt.

Strategy	How it works	When to use
Fixed-size	Split every N characters/tokens, with overlap.	Quick baseline; uniform text.
Recursive character	Try separators in order (\n\n, \n, '. ', ' ') so splits fall on paragraph/sentence boundaries. LangChain's RecursiveCharacterTextSplitter.	Default choice for most text.
Sentence-based	Split on sentences (nltk/spaCy) and group sentences up to a size.	Prose, articles.
Document-structure	Split by Markdown headers, HTML tags, code functions, PDF sections.	Docs with clear structure, code, manuals.
Semantic chunking	Embed sentences; start a new chunk where similarity between neighbouring sentences drops (topic change).	Mixed-topic docs; quality over speed.
Agentic / LLM-based	LLM decides boundaries or extracts self-contained 'propositions'.	High-value corpora; expensive.
Parent-child (small-to-big)	Index small child chunks for precise matching, return larger parent chunk for context.	Need both precision and context.
Sentence-window	Index single sentences; at retrieval expand to ±k surrounding sentences.	Fine-grained QA.

Chunk size and overlap — the trade-off

- **Too small** (e.g. 100 tokens): precise matches but loses context; answer may be split across chunks.

- **Too large** (e.g. 2000 tokens): more context but diluted embeddings, noisy retrieval, more tokens/cost.
- **Typical starting point:** 256–1024 tokens with 10–20% overlap. Overlap ensures a sentence cut at a boundary still appears whole in one chunk.
- Always tune using an evaluation set — there is no universally best size.

3.3 Embeddings

An **embedding model** converts text into a dense vector of numbers (e.g. 384, 768, 1024, 1536 or 3072 dimensions) such that texts with similar *meaning* lie close together in vector space. 'How do I reset my password?' and 'Forgot login credentials' end up near each other even with no shared words.

What happens inside

- Text → tokenizer → transformer encoder (e.g. BERT-like) → token embeddings → **pooling** (mean pooling or [CLS] token) → one vector → usually L2-normalised.
- Trained with **contrastive learning**: pull (query, relevant passage) pairs together, push irrelevant pairs apart.
- This is a **bi-encoder**: query and documents are encoded independently, so document vectors can be pre-computed and stored. Fast, scalable.

Similarity measures

Cosine similarity = (A · B) / (‖A‖ ‖B‖), range −1 to 1; measures angle (direction). **Dot product** = A · B; equals cosine when vectors are normalised. **Euclidean (L2) distance**: smaller = more similar. Use the metric the embedding model was trained for.

Popular embedding models

Type	Examples
Open-source (sentence-transformers)	all-MiniLM-L6-v2 (384-d, fast), all-mpnet-base-v2 (768-d), BAAI bge-small/base/large, e5, gte, nomic-embed
API-based	OpenAI text-embedding-3-small/large, Google Gemini embedding models, Cohere embed, Voyage AI
Sparse / learned sparse	BM25 (classical), SPLADE (learned sparse)
Late interaction	ColBERT — keeps one vector per token, matches token-to-token (MaxSim)

> **Watch out:** The query and the documents MUST be embedded with the same model. If you change the embedding model, you must re-embed the whole corpus. Some models (e5, bge) expect prefixes like 'query: ' and 'passage: '.

Choosing a model: check the **MTEB leaderboard** (retrieval tasks), language support, max input length, dimension (storage cost), latency, and whether data can leave your network.

3.4 Vector Database & Indexing

A vector database stores vectors + original text + metadata and answers the query: '*give me the k vectors closest to this one*'. Comparing against every vector (exact / brute-force kNN) is O(N) — too slow for millions — so we use **Approximate Nearest Neighbour (ANN)** indexes that trade a little accuracy for big speed gains.

Index	Idea	Notes
Flat (brute force)	Compare with every vector.	Exact, 100% recall; fine up to ~100k vectors.
IVF (Inverted File)	Cluster vectors with k-means; search only the nearest few clusters (nprobe).	Fast; recall depends on nprobe.
HNSW	Multi-layer 'small-world' graph; greedy walk from coarse top layer to fine bottom layer.	Most popular; high recall, fast; more memory.
PQ (Product Quantization)	Compress vectors into short codes.	Huge memory savings; some accuracy loss. Often IVF-PQ.

Vector store	Type / Notes
FAISS	Library by Meta (not a DB server); in-memory, very fast, many index types.
ChromaDB	Lightweight, easy local dev, persistent mode, metadata filters.
Pinecone	Fully managed cloud, serverless.
Weaviate / Qdrant / Milvus	Open-source servers; hybrid search, filtering, scaling.
pgvector	Vector extension for PostgreSQL — keep vectors next to relational data.
Elasticsearch / OpenSearch / Azure AI Search	Strong keyword (BM25) + vector = native hybrid search.

Metadata filtering: restrict search, e.g. *department* = '*HR*' *AND year* ≥ *2024*. Pre-filtering (filter then search) is accurate; post-filtering (search then filter) can return fewer than k results.

3.5 Retriever

The retriever takes the query and returns candidate chunks. Main strategies:

- **Dense retrieval**: embedding similarity (semantic meaning). Good for paraphrases; weak for exact IDs, codes, rare names.
- **Sparse / keyword retrieval (BM25, TF-IDF)**: exact term matching weighted by term rarity. Great for error codes, product SKUs, names; misses synonyms.
- **Hybrid retrieval**: both, merged (see Hybrid RAG).
- **MMR (Maximal Marginal Relevance)**: picks results that are relevant *and* diverse, avoiding 4 near-duplicate chunks. Score = λ·sim(q,d) − (1−λ)·max sim(d, already selected).
- **Similarity threshold**: only return chunks above a score; lets the system say 'I don't know' when nothing is relevant.
- **Top-k**: number of chunks to return. Higher k → better recall but more noise and tokens. Common pattern: retrieve 20–50, re-rank, keep 3–5.

BM25 in one line

BM25 scores a document higher when it contains the query terms often (term frequency, with saturation k1), when those terms are rare across the corpus (IDF), and normalises for document length (b). No neural network involved.

3.6 Query Transformation (pre-retrieval)

User queries are often short, vague, or conversational. Transforming them improves retrieval:

Technique	What happens
Query rewriting	LLM rewrites the query into a clear, search-friendly form (fix typos, expand acronyms).

Technique	What happens
Condensing (conversational)	Follow-up 'what about its price?' + chat history → standalone 'What is the price of iPhone 16?'
Multi-query	LLM generates 3–5 paraphrases; retrieve for each; union the results.
HyDE	LLM writes a hypothetical answer; embed that answer instead of the question (answer-to-answer matching).
Step-back prompting	Ask a more general question first ('What are the principles of X?') to fetch background context.
Query decomposition	Split complex question into sub-questions; retrieve for each; combine.
Query routing	Classify the query and send it to the right index/tool (HR docs vs IT docs vs SQL DB vs web).
Self-query / metadata extraction	LLM extracts filters from text: 'policies from 2024 about travel' → filter year=2024, query='travel policy'.

3.7 Re-ranking (post-retrieval)

The first-stage retriever is fast but approximate. A **re-ranker** re-scores the top candidates more accurately.

- **Cross-encoder**: feeds [query + chunk] *together* into one transformer, which outputs a relevance score. Because it sees both texts jointly (full attention across them), it is far more accurate than a bi-encoder — but too slow to run over millions of docs, so it is used only on the top 20–100.
- Examples: ms-marco-MiniLM cross-encoders, BAAI bge-reranker, Cohere Rerank, Jina reranker; or LLM-based reranking (ask the LLM to score/order).
- Classic two-stage pattern: **retrieve broadly (high recall) → re-rank precisely (high precision)**.

	Bi-encoder (embeddings)	Cross-encoder (re-ranker)
Input	Query and doc encoded separately	Query + doc encoded together
Pre-compute docs?	Yes — store vectors	No — must run per (query, doc) pair
Speed	Very fast (ANN search)	Slow
Accuracy	Good	Better
Role	Stage 1: candidate retrieval	Stage 2: re-ranking top-N

3.8 Context Construction & Compression

- **Deduplicate** overlapping chunks; **merge** adjacent chunks from the same document.
- **Ordering**: LLMs pay most attention to the beginning and end of the context ('**lost in the middle**' effect), so put the most relevant chunks first/last.
- **Contextual compression**: extract only relevant sentences from each chunk (LLM extractor or embedding filter) to save tokens and reduce noise.
- **Token budget**: make sure system prompt + context + history + question + expected answer fit in the context window.

3.9 Prompt Augmentation

The retrieved context is placed into a prompt template. A good RAG prompt:

- Defines the role and the task.
- Says to answer **only** from the provided context.
- Gives an explicit fallback: 'If the answer is not in the context, say I don't know.'
- Asks for citations (chunk IDs / source names).
- Clearly delimits context (e.g. XML-like tags or ### markers) to separate data from instructions.

Typical RAG prompt template

```
You are a helpful assistant for ACME HR.
Answer the question using ONLY the context below.
If the answer is not in the context, reply: "I don't know based on the available documents."
Cite sources as [source, page].

### CONTEXT
[1] (leave_policy.pdf, p.7) Employees are entitled to 12 sick leaves per year...
[2] (leave_policy.pdf, p.8) Unused sick leave cannot be carried forward...

### QUESTION
How many sick leaves do I get?
```

3.10 Generator (LLM)

The LLM reads the augmented prompt and produces the answer. Choices: GPT-4o/4.1, Gemini (Flash/Pro), Claude, Llama, Mistral, etc. Settings matter: a **low temperature (0–0.3)** for factual RAG; structured output (JSON) when needed; **streaming** to reduce perceived latency.

3.11 Supporting components

- **Memory / chat history**: for multi-turn conversations (condense question + history).
- **Caching**: embedding cache, retrieval cache, and **semantic cache** (reuse answers for semantically similar past questions).
- **Guardrails**: input filtering (prompt injection, PII), output checks (groundedness, toxicity).
- **Orchestration frameworks**: LangChain, LangGraph (stateful graphs, loops — ideal for CRAG/Self-RAG/agentic RAG), LlamaIndex (data/index-centric), Haystack, Semantic Kernel.
- **Observability**: LangSmith, Langfuse, Arize Phoenix — trace each retrieval and LLM call.
- **Evaluation**: RAGAS, DeepEval, TruLens (see Section 6).

4. Types of RAG — Explained One by One

RAG variants are not separate 'products' — they are **patterns** that improve a particular weakness of basic RAG (bad queries, missed keywords, irrelevant chunks, multi-step questions, relationships across documents, images, etc.). Real systems mix several of them. A useful mental map:

Category	Types	Fixes which weakness
Paradigms	Naive, Advanced, Modular	Overall maturity of the pipeline
Better retrieval	Hybrid, Multi-Query / RAG-Fusion, HyDE, Parent-Document / Sentence-Window, Contextual Retrieval, RAPTOR	Missing or poor-quality chunks
Self-checking / control flow	Corrective RAG, Self-RAG, Adaptive RAG, Iterative / Multi-hop RAG	Irrelevant context, hallucinations, complex questions
Autonomous	Agentic RAG, Multi-agent RAG	Need for tools, planning, multiple sources
Different data shapes	Graph RAG, Structured / SQL RAG, Multimodal RAG	Relationships, tables, images
Interaction	Conversational RAG	Follow-up questions with history

4.1 Naive (Simple / Vanilla) RAG

Core idea: The basic 'retrieve-then-read' pipeline: index → retrieve top-k by vector similarity → stuff into prompt → generate. No query rewriting, no re-ranking, no checks.



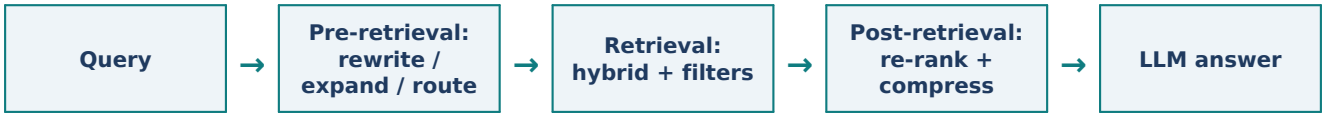
What happens inside

1. Documents are chunked with a fixed size and embedded once.
2. The raw user query is embedded with the same model.
3. Top-k nearest chunks by cosine similarity are returned (no matter how relevant they truly are).
4. Chunks are concatenated into the prompt ('stuff' strategy).
5. LLM generates in a single pass.

When to use	Pros	Cons
Prototypes, POCs, small clean corpora, simple factual Q&A.	Easy to build (~30 lines), cheap, low latency, easy to explain.	Low precision (irrelevant chunks), low recall (misses keyword matches), vague queries fail, no hallucination check, 'lost in the middle'.

4.2 Advanced RAG

Core idea: Naive RAG plus optimisations **before** retrieval (pre-retrieval) and **after** retrieval (post-retrieval) to improve quality.



What happens inside

1. **Indexing improvements**: better chunking (semantic/structural), richer metadata, cleaned data, sliding windows.
2. **Pre-retrieval**: query rewriting, expansion, HyDE, multi-query, routing, metadata extraction.
3. **Retrieval**: hybrid search, fine-tuned embeddings, metadata filtering.
4. **Post-retrieval**: cross-encoder re-ranking, context compression, deduplication, re-ordering.
5. **Generation**: tight prompts, citations, low temperature.

When to use	Pros	Cons
Most production RAG systems.	Much better precision/recall; still a linear, predictable pipeline.	More components to tune, more latency and cost.

4.3 Modular RAG

Core idea: Treat RAG as **interchangeable modules** (search, memory, routing, fusion, re-ranking, prediction, task adapters) that can be rearranged, including loops and branches — not only a fixed chain.



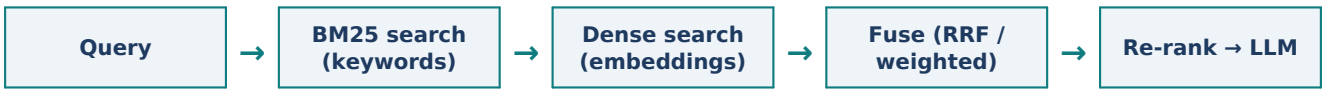
What happens inside

1. A router or orchestrator decides which modules to run for a query.
2. Modules can be swapped (e.g. Chroma → Qdrant, BM25 → SPLADE) without rewriting the system.
3. Patterns like retrieve→read, rewrite→retrieve→read, generate→read (HyDE), iterative retrieve-generate loops are all compositions of modules.
4. This is the paradigm that frameworks like LangGraph and LlamaIndex enable.

When to use	Pros	Cons
Large systems with many data sources and evolving requirements.	Flexible, maintainable, testable per module.	Design complexity; needs good orchestration and evaluation.

4.4 Hybrid RAG (Hybrid Search)

Core idea: Combine **sparse keyword search (BM25)** and **dense semantic search (embeddings)**, then fuse the results. Gets the exact-match strength of keywords and the meaning-match strength of vectors.



What happens inside

1. The same query runs in two retrievers in parallel.
2. BM25 catches exact tokens: 'error E1043', 'policy HR-17', product names, acronyms.
3. Dense search catches paraphrases: 'cannot log in' ≈ 'authentication failure'.
4. Results are merged with RRF or weighted score fusion; duplicates collapse.
5. Optionally a cross-encoder re-ranks the fused list.

Reciprocal Rank Fusion (RRF)

Scores from BM25 and cosine similarity are on different scales, so instead of adding raw scores we fuse **ranks**: RRF(d) = Σ over retrievers of 1 / (k + rank(d)), with k ≈ 60. A document ranked high by both retrievers wins. Alternative: **weighted score fusion** after normalising scores, e.g. 0.7·dense + 0.3·sparse (the 'alpha' parameter in Weaviate etc.).

RRF in Python

```
def rrf(result_lists, k=60):
    scores = {}
    for results in result_lists:                    # e.g. [bm25_ids, dense_ids]
        for rank, doc_id in enumerate(results, start=1):
            scores[doc_id] = scores.get(doc_id, 0) + 1 / (k + rank)
    return sorted(scores, key=scores.get, reverse=True)
```

When to use	Pros	Cons
Enterprise docs, technical/IT tickets, legal, e-commerce search — anywhere with IDs, codes, jargon.	Consistently better recall than either alone; robust to query style.	Two indexes to maintain; weight tuning; slightly more latency.

In LangChain: *EnsembleRetriever([bm25_retriever, vector_retriever], weights=[0.4, 0.6])* implements weighted RRF.

4.5 Multi-Query RAG & RAG-Fusion

Core idea: One query phrasing may miss relevant chunks. Generate several reformulations with an LLM, retrieve for each, then merge. **RAG-Fusion** = multi-query + RRF ranking of the merged results.



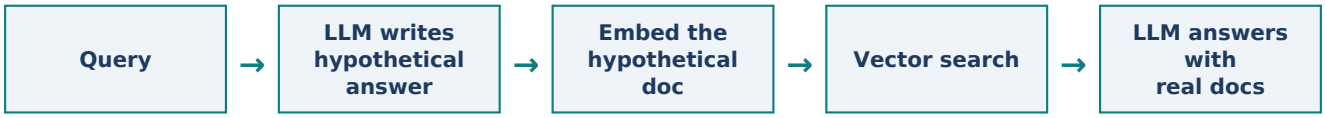
What happens inside

1. LLM prompt: 'Generate 4 different versions of this question to retrieve relevant documents.'
2. Each variant is embedded and searched independently (can run in parallel).
3. Results are unioned (multi-query) or rank-fused with RRF (RAG-Fusion), so chunks found by many variants rise to the top.
4. Top fused chunks go to the LLM.

When to use	Pros	Cons
Ambiguous or short queries; users who phrase things differently from the docs.	Higher recall; covers multiple perspectives.	Extra LLM call + N retrievals → more latency/cost; can drift off-topic.

4.6 HyDE (Hypothetical Document Embeddings)

Core idea: Questions and answers look different in vector space. So ask the LLM to write a **fake (hypothetical) answer** first, embed that, and search with it — answer-to-answer similarity is stronger than question-to-answer.



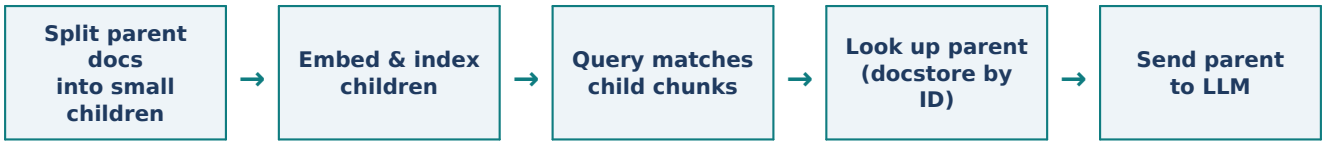
What happens inside

1. Query: 'What causes HNSW memory growth?' → LLM drafts a paragraph that sounds like documentation.
2. The draft may contain factual errors — that is fine; we only use it to find *similar real* documents.
3. The draft's embedding lands closer to real relevant passages than the short question would.
4. The final answer is generated only from the real retrieved documents.

When to use	Pros	Cons
Short queries, zero-shot domains without fine-tuned embeddings.	Simple, often big recall boost.	Extra LLM call; if the LLM knows nothing about the domain, the fake answer can mislead retrieval.

4.7 Parent-Document, Small-to-Big & Sentence-Window RAG

Core idea: Small chunks give precise matches; big chunks give enough context. Decouple them: **search on small pieces, return the bigger piece**.



What happens inside

1. Each document is split into large **parent** chunks (e.g. 2000 tokens), each split into small **child** chunks (e.g. 200 tokens).
2. Only children are embedded; each child stores its parent_id in metadata.
3. At query time the precise child match is found, then the parent is fetched from a docstore and passed to the LLM.
4. **Sentence-window** variant: index individual sentences and return a window of ±k sentences around the hit.
5. **Auto-merging** (LlamaIndex): if many children of the same parent are retrieved, replace them with the parent.

When to use	Pros	Cons
Long technical docs, contracts, manuals where an answer needs surrounding context.	Precise retrieval + rich context.	Needs a docstore + ID mapping; more tokens per retrieved hit.

4.8 Contextual Retrieval (Contextual RAG)

Core idea: A chunk like 'Revenue grew 3% over the previous quarter' has lost its context (which company? which quarter?). Before embedding, use an LLM to **prepend a short context** to every chunk explaining where it comes from.



What happens inside

1. For each chunk, the LLM sees the whole document and writes e.g. 'This chunk is from ACME's Q2 2024 10-Q filing, discussing revenue.'
2. This context is prepended and the combined text is embedded (contextual embeddings) and BM25-indexed (contextual BM25).

3. Prompt caching keeps the cost of passing the full document many times manageable.
4. Typically combined with hybrid search and re-ranking (popularised by Anthropic).

When to use	Pros	Cons
Corpora where chunks are ambiguous out of context: financial reports, legal docs, long manuals.	Significantly fewer retrieval failures; works with any retriever.	Indexing cost (one LLM call per chunk); slower ingestion.

4.9 RAPTOR (Hierarchical / Tree-Summarisation RAG)

Core idea: Build a **tree of summaries**: cluster chunks, summarise each cluster, cluster the summaries, summarise again... Retrieval can then hit both detailed leaves and high-level summaries.



What happens inside

1. Chunks are embedded and clustered (e.g. Gaussian Mixture Models).
2. Each cluster is summarised by an LLM → new 'summary nodes', which are embedded too.
3. The process repeats to build higher abstraction levels.
4. At query time, search the collapsed tree (all levels) so broad questions match summaries and specific ones match leaves.

When to use	Pros	Cons
Questions needing a holistic understanding of long documents ('What is the main theme of this book?').	Handles both broad and specific questions.	Expensive indexing; must rebuild summaries when data changes.

4.10 Conversational RAG (RAG with Memory)

Core idea: In chat, follow-ups like 'and for interns?' cannot be retrieved on their own. Use chat history to create a **standalone question** before retrieval.



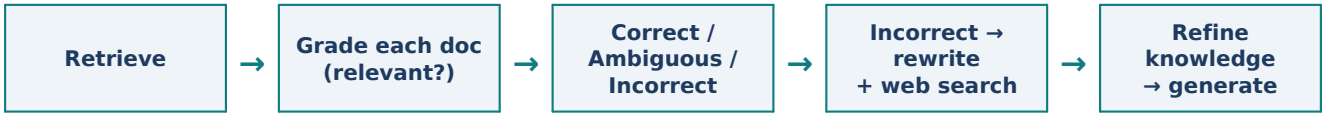
What happens inside

1. History: 'How many sick leaves do employees get?' → 'And for interns?'
2. Condense step → 'How many sick leaves do interns get?'
3. Retrieval uses the standalone question; generation sees context + recent history.
4. Memory strategies: last N turns (window), summary memory, or retrieval over past turns.
5. In LangChain: create_history_aware_retriever; in LangGraph: messages in state + a checkpointer (thread_id).

When to use	Pros	Cons
Chatbots, support assistants, any multi-turn interface.	Natural conversations; correct retrieval on follow-ups.	Extra LLM call; long histories cost tokens; condensing can lose intent.

4.11 Corrective RAG (CRAG)

Core idea: Don't blindly trust retrieval. A **retrieval evaluator** grades the retrieved documents; if they are poor, the system **corrects** itself — rewriting the query and/or falling back to web search — before generating.



What happens inside

1. Retrieve top-k chunks as usual.
2. A lightweight evaluator (small model or LLM with a yes/no structured output) scores each chunk's relevance.
3. **Correct** (confident relevant): keep and refine — strip irrelevant sentences ('decompose-then-recompose').
4. **Incorrect**: discard and go to web search (e.g. Tavily) with a rewritten query.
5. **Ambiguous**: combine refined internal docs + web results.
6. Generate the final answer from the corrected knowledge.

CRAG in LangGraph (skeleton)

Corrective RAG graph

```
class State(TypedDict): question: str; docs: list; web_needed: bool; answer: str

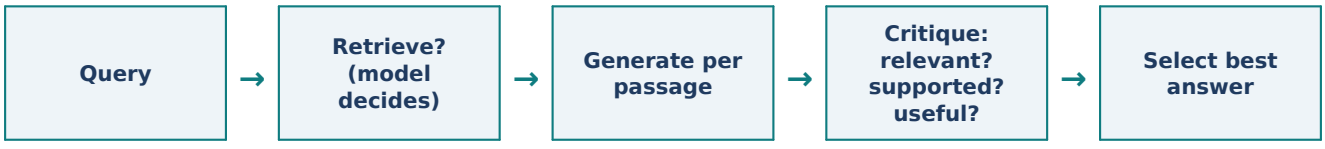
def retrieve(s): return {"docs": retriever.invoke(s["question"])}
def grade(s):
    good = [d for d in s["docs"]
            if grader.invoke({"q": s["question"], "doc": d.page_content}).score == "yes"]
    return {"docs": good, "web_needed": len(good) == 0}
def rewrite(s): return {"question": rewriter.invoke(s["question"])}
def web_search(s): return {"docs": s["docs"] + tavily.invoke(s["question"])}
def generate(s):
    return {"answer": rag_chain.invoke({"context": s["docs"], "question": s["question"]})}

g = StateGraph(State)
nodes = {"retrieve": retrieve, "grade": grade, "rewrite": rewrite, "web": web_search, "generate": generate}
for name, fn in nodes.items(): g.add_node(name, fn)
g.add_edge(START, "retrieve"); g.add_edge("retrieve", "grade")
g.add_conditional_edges("grade", lambda s: "rewrite" if s["web_needed"] else "generate")
g.add_edge("rewrite", "web"); g.add_edge("web", "generate"); g.add_edge("generate", END)
app = g.compile()
```

When to use	Pros	Cons
When the knowledge base may be incomplete; open-domain assistants; high-accuracy Q&A.	Robust to bad retrieval; fewer hallucinations; graceful fallback.	Extra grading calls (latency, cost); web data can be noisy/untrusted.

4.12 Self-RAG (Self-Reflective RAG)

Core idea: The model itself decides **whether to retrieve**, and **critiques** both the retrieved passages and its own output using special **reflection tokens**, then picks the best supported answer.



What happens inside

1. Original paper (Asai et al., 2023) trains an LM to emit reflection tokens: **[Retrieve]** (needed?), **[IsREL]** (is passage relevant?), **[IsSUP]** (is the output supported by the passage?), **[IsUSE]** (is the response useful?).
2. If retrieval is not needed (e.g. 'write a poem'), it answers directly.
3. Otherwise it generates candidate segments for each passage and scores them with the critique tokens; the best-scoring segment wins.
4. In practice (LangGraph implementations) the reflection tokens are replaced by LLM **graders**: document-relevance grader → generate → **hallucination grader** (is the answer grounded?) → **answer grader** (does it address the question?) → loop back to rewrite/regenerate if not.

When to use	Pros	Cons
High-stakes answers where faithfulness must be verified (legal, medical, finance, compliance).	Self-verification, adaptive retrieval, fewer hallucinations.	Many LLM calls, higher latency; loops need max-iteration limits.

Interview tip: CRAG vs Self-RAG: CRAG focuses on *fixing the retrieved knowledge* (grade docs → fall back to web). Self-RAG focuses on *reflecting on the whole process* — whether to retrieve, and whether the generated answer is supported and useful.

4.13 Adaptive RAG

Core idea: Not every question needs the same pipeline. A **query classifier/router** estimates complexity and picks the strategy: no retrieval, single-step RAG, or multi-step RAG (or web search).



What happens inside

1. A small classifier or LLM with structured output labels the query (e.g. A: no retrieval, B: single-step, C: multi-step).
2. 'What is 2+2?' → answer directly; 'What is our refund policy?' → single retrieval; 'Compare refund policies of 2022 and 2024 and explain the impact' → iterative retrieval.
3. Often combined with routing to different sources: vector store vs web search.
4. Popular LangGraph example: router → (vectorstore → CRAG/Self-RAG checks) or (web search).

When to use	Pros	Cons
Mixed traffic with both trivial and complex questions.	Saves cost/latency on simple queries; better on hard ones.	Classifier errors route queries wrongly; more design work.

4.14 Iterative / Multi-hop RAG

Core idea: Some questions need facts that depend on other facts: 'Who is the manager of the person who wrote the leave policy?' Retrieve → reason → retrieve again using what was learned, until enough information is gathered.



What happens inside

1. Decompose: Hop 1 'Who wrote the leave policy?' → 'Priya'.
2. Hop 2 uses the answer: 'Who is Priya's manager?' → 'Rahul'.
3. Techniques: IRCoT (interleave retrieval with chain-of-thought), ITER-RETGEN (use the generation to retrieve again), FLARE (retrieve when the model's next sentence has low-confidence tokens).
4. Stop condition: model judges info sufficient or max hops reached.

When to use	Pros	Cons
Multi-hop QA, research questions, comparisons across documents.	Answers questions single-shot RAG cannot.	Latency grows with hops; error in hop 1 propagates.

4.15 Agentic RAG

Core idea: An **LLM agent** (ReAct: Reason + Act loop) treats retrieval as one of several **tools** and autonomously plans: which source to query, whether to search again, whether to call a calculator/SQL/API, and when it is done.



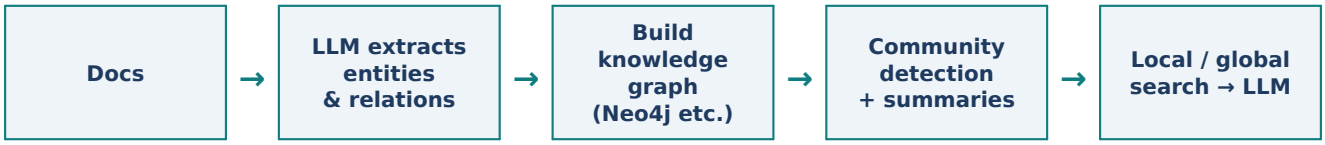
What happens inside

1. Tools are described to the LLM (name, description, arguments); e.g. search_hr_docs, search_it_docs, web_search, run_sql.
2. Agent loop: Thought → Action (tool call) → Observation → Thought ... (ReAct).
3. It can reformulate queries, query multiple indexes, cross-check, and decide when it has enough.
4. **Multi-agent RAG**: a supervisor routes to specialist agents (researcher, retriever, SQL agent, writer, reviewer).
5. Built with LangGraph (create_react_agent / custom graphs), LlamaIndex agents, CrewAI, AutoGen, OpenAI/Google agent SDKs.

When to use	Pros	Cons
Many heterogeneous sources, open-ended tasks, research assistants, customer support needing actions.	Most flexible and capable; handles complex, multi-source tasks.	Less predictable, higher cost/latency, harder to evaluate/debug; needs guardrails and step limits.

4.16 Graph RAG (Knowledge-Graph RAG)

Core idea: Vector search finds similar *text* but not *relationships*. Graph RAG extracts **entities and relationships** into a knowledge graph and retrieves by traversing it — great for 'connect the dots' and global questions.



What happens inside

1. Indexing: an LLM extracts triples like (Priya) —[MANAGES]→ (Payments Team), (Payments Team) —[OWNS]→ (Refund API).
2. Stored in a graph DB (Neo4j, Memgraph) or in-memory graph.

3. Microsoft GraphRAG additionally clusters the graph into **communities** (Leiden algorithm) and pre-writes a summary for each community.
4. **Local search**: find entities in the query, traverse their neighbourhood, collect related text.
5. **Global search**: map-reduce over community summaries to answer 'What are the main themes across all incident reports?'
6. Alternative: Text-to-Cypher — LLM writes a graph query.

When to use	Pros	Cons
Highly connected data: org charts, supply chains, fraud, research literature, incident root-cause analysis, holistic 'summarise everything' questions.	Multi-hop reasoning, explainable paths, global understanding.	Expensive and complex indexing; extraction errors; harder to update.

4.17 Structured / SQL RAG (Text-to-SQL)

Core idea: When knowledge lives in tables (sales, orders, tickets), similarity search is the wrong tool. The LLM **generates a query** (SQL/Pandas/Cypher), executes it, and explains the result.



What happens inside

1. Relevant table schemas, column descriptions and few-shot example queries are retrieved (schema RAG) and placed into the prompt.
2. LLM generates SQL: 'SELECT COUNT(*) FROM tickets WHERE priority='P1' AND created_at >= ...'.
3. Query is validated and run with a **read-only** user; errors can be sent back for self-correction.
4. Result rows are summarised in natural language. Hybrid systems route between SQL and document RAG.

When to use	Pros	Cons
Analytics questions, aggregations, numbers ('How many', 'total', 'average').	Exact numeric answers; uses existing databases.	SQL errors, security risk (SQL injection, data access) — needs strict permissions.

4.18 Multimodal RAG

Core idea: Retrieve and reason over **images, tables, charts, audio, video** — not only text.



What happens inside

1. Option A: **shared embedding space** (CLIP, SigLIP, multimodal embedding models) — embed images and text into the same space and search across them.
2. Option B: **summarise to text** — a vision LLM captions every image/table; the caption is embedded, but the original image is returned to the generator (multi-vector retriever).
3. Option C: **page-image retrieval** (e.g. ColPali) — embed screenshots of whole PDF pages; no OCR/parsing needed.

4. A multimodal LLM reads the retrieved text + images to answer.

When to use	Pros	Cons
Manuals with diagrams, slide decks, invoices, medical images, product catalogues.	Uses information plain-text RAG throws away.	Expensive storage and inference; harder evaluation.

4.19 Other names you may hear

Name	One-line meaning
Speculative RAG	A small 'drafter' model writes multiple drafts from different document subsets; a large 'verifier' model picks the best — faster and accurate.
Branched / Routing RAG	Router sends the query to one of several specialised indexes or pipelines.
Federated RAG	Retrieves from multiple distributed/private data sources without centralising data.
Long-context RAG / LongRAG	Retrieve larger units (whole sections/documents) because modern LLMs have big windows.
Memory RAG	Long-term memory of user facts stored in a vector store and retrieved per conversation.
Cache-Augmented Generation (CAG)	Alternative to RAG: preload a small knowledge base into the model's KV cache; no retrieval step.

4.20 Summary comparison of RAG types

Type	Key mechanism	Extra LLM calls	Complexity	Best for
Naive	Embed → top-k → generate	0	Low	Prototypes
Advanced	Pre + post retrieval optimisation	0–2	Medium	Production baseline
Modular	Swappable modules, routing, loops	Varies	Medium–High	Large systems
Hybrid	BM25 + dense + RRF	0	Low–Medium	IDs, codes, jargon
Multi-Query / Fusion	N query variants + RRF	1	Low	Vague queries
HyDE	Embed a fake answer	1	Low	Short queries
Parent-Document	Search small, return big	0	Low–Medium	Long docs
Contextual	LLM context prepended per chunk	1 per chunk (index)	Medium	Ambiguous chunks
RAPTOR	Tree of cluster summaries	Many (index)	High	Holistic questions
Conversational	Condense with history	1	Low	Chatbots
Corrective (CRAG)	Grade docs → web fallback	k + 1–2	Medium	Incomplete KB
Self-RAG	Reflect: retrieve? relevant? supported?	Several	High	High-stakes accuracy
Adaptive	Route by complexity	1	Medium	Mixed traffic
Multi-hop	Iterative retrieve-reason	Several	High	Multi-step questions
Agentic	Agent + tools (ReAct)	Many	High	Multi-source tasks
Graph	Knowledge graph traversal	Many (index)	High	Relationships, global Qs

Type	Key mechanism	Extra LLM calls	Complexity	Best for
SQL / Structured	Text-to-SQL + execute	1–2	Medium	Numeric / tabular
Multimodal	Image/table embeddings	Varies	High	Visual docs

5. Hands-on: Building RAG in Python

Three compact implementations, from scratch to framework, using tools you already work with (sentence-transformers, ChromaDB, Gemini, LangChain).

5.1 Naive RAG from scratch (no framework)

rag_basic.py

```
# pip install sentence-transformers chromadb google-genai
import chromadb
from sentence_transformers import SentenceTransformer
from google import genai

embedder = SentenceTransformer("all-MiniLM-L6-v2")
db = chromadb.PersistentClient(path="./chroma_db")
col = db.get_or_create_collection("docs", metadata={"hnsw:space": "cosine"})
llm = genai.Client()                # reads GEMINI_API_KEY / GOOGLE_API_KEY from env

def chunk(text, size=500, overlap=50):
    return [text[i:i + size] for i in range(0, len(text), size - overlap)]

def index(doc_id, text, source):
    chunks = chunk(text)
    col.add(ids=[f"{doc_id}_{i}" for i in range(len(chunks))], documents=chunks,
            embeddings=embedder.encode(chunks).tolist(),
            metadatas=[{"source": source}] * len(chunks))

def retrieve(query, k=4):
    res = col.query(query_embeddings=embedder.encode([query]).tolist(), n_results=k)
    return list(zip(res["documents"][0], res["metadatas"][0], res["distances"][0]))

def answer(query):
    hits = retrieve(query)
    context = "\n\n".join(f"[{i+1}] ({m['source']}) {d}" for i, (d, m, _) in enumerate(hits))
    prompt = f"""Answer ONLY from the context. If not found, say "I don't know". Cite [n].
Context:
{context}

Question: {query}"""
    return llm.models.generate_content(model="gemini-2.5-flash", contents=prompt).text
```

5.2 Hybrid retrieval (BM25 + dense) with RRF

hybrid.py

```
# pip install rank_bm25
from rank_bm25 import BM25Okapi
import numpy as np

corpus = [...]                                          # list of chunk strings
bm25 = BM25Okapi([c.lower().split() for c in corpus])
doc_vecs = embedder.encode(corpus, normalize_embeddings=True)

def hybrid_search(query, k=5, pool=20, rrf_k=60):
    sparse = np.argsort(bm25.get_scores(query.lower().split()))[::-1][:pool]
    dense = np.argsort(doc_vecs @ embedder.encode(query, normalize_embeddings=True))[::-1][:pool]
    scores = {}
    for ranking in (sparse, dense):
        for rank, idx in enumerate(ranking, 1):
            scores[idx] = scores.get(idx, 0) + 1 / (rrf_k + rank)
    return [corpus[i] for i in sorted(scores, key=scores.get, reverse=True)[:k]]
```

5.3 Re-ranking with a cross-encoder

rerank.py

```
from sentence_transformers import CrossEncoder
reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def rerank(query, docs, top_n=3):
    scores = reranker.predict([(query, d) for d in docs])
    return [d for _, d in sorted(zip(scores, docs), reverse=True)[:top_n]]

final_docs = rerank(q, hybrid_search(q, k=20))
```

5.4 The same pipeline with LangChain (LCEL)

rag_langchain.py

```
# pip install langchain langchain-community langchain-chroma langchain-huggingface
#             langchain-google-genai pypdf
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_chroma import Chroma
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

docs = PyPDFLoader("policy.pdf").load()
chunks = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=100).split_documents(docs)
emb = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vs = Chroma.from_documents(chunks, emb, persist_directory="db")
retriever = vs.as_retriever(search_type="mmr", search_kwargs={"k": 4, "fetch_k": 20})

prompt = ChatPromptTemplate.from_template(
    "Answer only from context. If unknown, say so.\nContext:\n{context}\n\nQuestion: {question}")
fmt = lambda ds: "\n\n".join(f"[{d.metadata.get('page')}] {d.page_content}" for d in ds)
llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0)

chain = ({"context": retriever | fmt, "question": RunnablePassthrough()}
         | prompt | llm | StrOutputParser())
print(chain.invoke("How many sick leaves do I get?"))
```

Watch out: Libraries change fast — package names and model IDs (e.g. Gemini model versions) get updated. If an import fails, check the current docs for the package version you installed.

6. Evaluating RAG

You must evaluate the **retriever** and the **generator** separately — otherwise you cannot tell which one is failing.

6.1 Retrieval metrics (need labelled relevant chunks per query)

Metric	Meaning	Formula / intuition
Hit Rate@k	Did at least one relevant chunk appear in top-k?	Fraction of queries with ≥1 hit
Recall@k	Of all relevant chunks, how many did we retrieve in top-k?	relevant retrieved / total relevant
Precision@k	Of the k retrieved, how many are relevant?	relevant retrieved / k
MRR	How high is the first relevant result?	mean of 1 / rank of first relevant
nDCG@k	Rewards relevant results at higher ranks; supports graded relevance	DCG / ideal DCG
MAP	Mean of average precision across queries	Order-aware precision

6.2 Generation / end-to-end metrics (RAG Triad and RAGAS)

Metric	Question it answers	Compares
Faithfulness / Groundedness	Is every claim in the answer supported by the retrieved context? (hallucination check)	Answer ↔ Context
Answer relevancy	Does the answer actually address the question?	Answer ↔ Question
Context precision	Are the relevant chunks ranked at the top? Is the context free of noise?	Context ↔ Question / ground truth
Context recall	Does the context contain all the info needed for the ground-truth answer?	Context ↔ Ground truth
Answer correctness	Is the answer factually equal to the reference answer?	Answer ↔ Ground truth

> **Key idea: RAG Triad** (TruLens): Context relevance (query↔context), Groundedness (context↔answer), Answer relevance (query↔answer). If all three are high, the system is unlikely to hallucinate.

6.3 How to evaluate in practice

1. Build a **golden dataset**: 50–200 real questions with reference answers and source chunks (manually, or synthetically generated with an LLM and then reviewed).
2. Run retrieval metrics first; fix chunking/embeddings/hybrid/reranking until recall@k is good.
3. Run generation metrics with **LLM-as-a-judge** (RAGAS, DeepEval, TruLens, LangSmith evaluators).
4. Track latency (p50/p95), cost per query, and token usage.
5. Add **human evaluation** and user feedback (thumbs up/down) in production; feed failures back into the test set.
6. Run evaluation in CI on every change (prompt, chunk size, model) — regression testing for RAG.

RAGAS sketch (API varies by version)

```
# pip install ragas datasets
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision, context_recall
from datasets import Dataset

data = Dataset.from_dict({"question": qs, "answer": answers,
                          "contexts": contexts, "ground_truth": refs})
print(evaluate(data, metrics=[faithfulness, answer_relevancy, context_precision, context_recall]))
```

7. Failure Modes, Fixes & Production Concerns

7.1 Common failure points and fixes

Symptom	Likely cause	Fix
Right doc exists but not retrieved	Bad chunking, weak embeddings, vocabulary mismatch	Hybrid search, better chunking, query rewriting/HyDE, domain/fine-tuned embeddings
Retrieved but ranked low (cut off by top-k)	First-stage retriever imprecise	Retrieve more (k=20–50) + cross-encoder rerank
Fails on IDs, codes, names	Dense-only retrieval	Add BM25 (hybrid), metadata filters
Answer ignores the context / hallucinates	Weak prompt, noisy context, high temperature	Strict grounding prompt, compression, temp 0, faithfulness checks (Self-RAG)
Answer is incomplete	Info spread across chunks; k too small	Parent-document retrieval, larger k, multi-hop / decomposition
Correct chunk but wrong answer	Lost in the middle; too much context	Reorder, fewer but better chunks
Outdated answers	Stale index; old versions of docs	Incremental re-indexing, date metadata, filter/boost recent
Follow-up questions fail	No history in retrieval	Conversational RAG (condense question)
Wrong format	No format instructions	Structured output / JSON schema
Slow responses	Many LLM calls, large k, reranker	Caching, streaming, smaller models for grading, async/parallel retrieval

7.2 Production checklist

- **Data pipeline**: incremental ingestion (only new/changed docs, via content hashes), deletion handling, versioning, scheduled re-index.
- **Security / access control**: store permissions (ACLs) as metadata and filter at query time so users only retrieve documents they are allowed to see. Never rely on the LLM to hide data.
- **Prompt injection**: retrieved documents may contain instructions ('ignore previous instructions...'). Delimit context, tell the model to treat it as data, sanitise sources, limit tool permissions.
- **PII & compliance**: mask sensitive data before indexing; data residency for embedding/LLM APIs.
- **Latency**: stream tokens, parallelise retrievers, cache embeddings and frequent answers (semantic cache), use smaller models for routing/grading.
- **Cost**: fewer/shorter chunks, cheaper embedding model, batch embedding, prompt caching.
- **Scale**: ANN index choice (HNSW/IVF-PQ), sharding, managed vector DB.
- **Observability**: log query, retrieved chunk IDs + scores, prompt, answer, latency, cost; trace with LangSmith/Langfuse/Phoenix.
- **Fallbacks**: 'I don't know' path, human handoff, web search fallback.
- **Feedback loop**: collect user feedback, review failures weekly, add to evaluation set.

8. Interview Questions & Answers

Answers are written to be **speakable** in an interview: start with a one-line definition, then the 'why', then an example or trade-off.

8.1 Basics

Q1. What is RAG?

RAG is an architecture where we retrieve relevant information from an external knowledge source and add it to the LLM's prompt so the answer is grounded in that information. It has an indexing pipeline (load, chunk, embed, store) and a query pipeline (embed query, retrieve, augment prompt, generate).

Q2. Why use RAG instead of just asking the LLM?

LLMs have a knowledge cutoff, don't know private data, and hallucinate. RAG gives them fresh, domain-specific, citable context at query time without retraining.

Q3. RAG vs fine-tuning — when do you choose which?

RAG for knowledge that is large, private or frequently changing, and when citations are needed. Fine-tuning for behaviour: style, format, domain language, or a specialised task. They can be combined. Updating RAG is just re-indexing; updating fine-tuning requires retraining.

Q4. With 1M+ token context windows, is RAG still needed?

Yes, for most real systems. Corpora are often far larger than any window; long prompts cost more and are slower on every call; models suffer 'lost in the middle'; and RAG gives access control, freshness and citations. Long context does reduce the need for very aggressive chunking, and for small static corpora CAG/long-context can be simpler.

Q5. Explain the components of a RAG pipeline.

Document loader and parser, cleaner, chunker, embedding model, vector store (with ANN index and metadata), retriever, optional query transformer and re-ranker, prompt template, LLM generator, plus memory, caching, guardrails and evaluation.

Q6. What does 'grounding' mean?

Making the LLM base its answer on supplied evidence rather than its parametric memory, ideally with citations to that evidence. Faithfulness measures how well the answer is grounded.

Q7. What is parametric vs non-parametric memory?

Parametric memory is knowledge stored in model weights. Non-parametric memory is the external index the retriever searches. RAG combines both — the original RAG paper used exactly these terms.

8.2 Chunking & Embeddings

Q8. Why do we chunk documents?

Embedding models have input limits; one vector for a long document blurs its meaning; and smaller chunks let us send only relevant text to the LLM, reducing noise and cost.

Q9. How do you choose chunk size and overlap?

Start around 256–1024 tokens with 10–20% overlap, adapted to the document type and embedding model limit, then tune on an evaluation set by measuring recall@k and answer quality. Small chunks are precise but lack context; large chunks have context but dilute the embedding.

Q10. What is semantic chunking?

Splitting where the meaning changes: embed consecutive sentences and break the chunk where the similarity between neighbours drops below a threshold, so each chunk stays on one topic.

Q11. What is an embedding?

A dense numerical vector representation of text where semantically similar texts are close together. It is produced by a transformer encoder followed by pooling, trained with contrastive learning.

Q12. Cosine similarity vs dot product vs Euclidean?

Cosine measures the angle between vectors (ignores magnitude). Dot product includes magnitude; for normalised vectors it equals cosine. Euclidean measures straight-line distance. Use what the embedding model was trained with; with normalised vectors all give the same ranking.

Q13. Bi-encoder vs cross-encoder?

A bi-encoder embeds query and document separately, so document vectors can be precomputed — fast, used for retrieval. A cross-encoder processes query and document together and outputs a relevance score — more accurate but slow, so it is used to re-rank the top candidates.

Q14. How do you choose an embedding model?

Check MTEB retrieval scores, language and domain fit, max input length, vector dimension (storage and speed), latency, cost, and data-privacy needs (API vs self-hosted). Then validate on your own golden dataset.

Q15. What happens if you change the embedding model?

All documents must be re-embedded, because vectors from different models live in different spaces and are not comparable.

Q16. When would you fine-tune an embedding model?

When the domain vocabulary is specialised (legal, medical, internal jargon) and off-the-shelf recall is poor. Train on (query, relevant passage) pairs — often synthetic pairs generated by an LLM — with a contrastive loss such as MultipleNegativesRankingLoss.

8.3 Retrieval & Vector DBs

Q17. What is a vector database and why not a normal DB?

It stores vectors with metadata and supports fast similarity search via ANN indexes. Normal databases index exact values, not 'closest in high-dimensional space'. (pgvector adds this to Postgres.)

Q18. What is ANN and why do we need it?

Approximate Nearest Neighbour search finds very close (not guaranteed exact) neighbours much faster than comparing with every vector. Brute force is O(N) per query, which is too slow for millions of vectors.

Q19. Explain HNSW.

Hierarchical Navigable Small World: a multi-layer graph where the top layers have few long-range links and lower layers have many short links. Search starts at the top, greedily moves to the closest node, and descends layer by layer. Parameters: M (links per node), efConstruction, efSearch (recall vs speed).

Q20. Explain IVF and PQ.

IVF clusters vectors with k-means and searches only the nprobe nearest clusters. PQ splits each vector into sub-vectors and replaces each with a codebook ID, compressing memory heavily. IVF-PQ combines both for billion-scale search.

Q21. What is hybrid search and why does it help?

Combining BM25 keyword search with dense vector search and fusing the results, often with Reciprocal Rank Fusion. Dense handles synonyms and paraphrases; BM25 handles exact terms like error codes and names. Together they improve recall.

Q22. What is Reciprocal Rank Fusion?

A rank-based way to merge result lists: each document gets the sum of 1/(k + rank) over all lists, with k usually 60. It avoids normalising incompatible scores and rewards documents that rank well in several lists.

Q23. What is MMR?

Maximal Marginal Relevance selects documents that are relevant to the query but not redundant with already-selected ones, using a λ parameter to balance relevance and diversity.

Q24. How do you choose top-k?

Balance recall against noise and token cost. A common pattern is to retrieve 20–50 candidates, re-rank, and keep the top 3–5. Tune using recall@k and answer quality; consider a similarity threshold to allow 'no answer'.

Q25. What is re-ranking and where does it sit?

A second stage after retrieval where a more accurate model (cross-encoder, Cohere Rerank, or LLM) re-scores the candidate chunks. It boosts precision so the best chunks reach the prompt.

Q26. What is metadata filtering? Pre vs post filtering?

Restricting search by attributes like department, date or user permissions. Pre-filtering applies the filter during the search (correct k results); post-filtering filters after retrieval and may leave fewer than k results.

Q27. Explain HyDE.

Hypothetical Document Embeddings: the LLM writes a hypothetical answer to the query, and we embed that answer to search, because answer-to-document similarity is stronger than question-to-document similarity. The final answer still uses only the real retrieved documents.

Q28. What is the 'lost in the middle' problem?

LLMs use information at the start and end of long contexts better than information in the middle. Mitigate by sending fewer, re-ranked chunks and placing the most relevant at the beginning or end.

8.4 Advanced RAG types

Q29. Naive vs Advanced vs Modular RAG?

Naive is a fixed retrieve-then-generate chain. Advanced adds pre-retrieval (query rewriting, routing) and post-retrieval (re-ranking, compression) optimisations. Modular breaks RAG into swappable modules with routing, loops and branches, orchestrated by tools like LangGraph.

Q30. Explain Corrective RAG.

After retrieval, an evaluator grades document relevance. If documents are relevant, they are refined and used; if irrelevant, the query is rewritten and a web search supplies new knowledge; if ambiguous, both are combined. Then the answer is generated. It makes RAG robust to bad retrieval.

Q31. Explain Self-RAG.

The model decides when to retrieve and critiques itself using reflection tokens: is the passage relevant, is the answer supported, is it useful. In LangGraph it is implemented with graders: document grader, hallucination grader and answer grader, looping back to rewrite or regenerate if checks fail.

Q32. Difference between CRAG and Self-RAG?

CRAG corrects the retrieved knowledge, mainly with a relevance grader and web-search fallback. Self-RAG reflects on the whole process — whether retrieval is needed and whether the generation is grounded and useful — and can loop on generation.

Q33. What is Adaptive RAG?

A router classifies query complexity and chooses the strategy: answer directly, single-step retrieval, or multi-step retrieval / web search. It saves cost on easy questions and improves quality on hard ones.

Q34. What is Agentic RAG?

Retrieval becomes a tool used by an LLM agent in a ReAct loop. The agent plans, chooses which tools or indexes to call, evaluates observations, retries with new queries, and stops when it has enough. It is more capable but less predictable and costlier, so it needs step limits and guardrails.

Q35. What is Graph RAG and when is it better than vector RAG?

It extracts entities and relationships into a knowledge graph and retrieves by traversing it, optionally with community summaries (Microsoft GraphRAG). It is better for multi-hop relationship questions and global questions like 'what are the main themes across all documents', where top-k similar chunks are not enough.

Q36. What is multi-hop RAG?

Answering questions that require chaining facts: retrieve, extract an intermediate answer, form a new query, retrieve again, until enough evidence is collected. Examples: IRCoT, FLARE.

Q37. How do you handle tables and images in RAG?

Use layout-aware parsers; convert tables to Markdown or summarise them; caption images with a vision model and embed the captions while returning the original image (multi-vector retrieval); or use multimodal embeddings / page-image retrieval like ColPali, and a multimodal LLM for generation.

Q38. What is the parent-document retriever?

Index small child chunks for precise matching, but return the larger parent chunk to the LLM so it has enough context. Children store their parent ID; parents live in a docstore.

Q39. How does conversational RAG handle follow-up questions?

It uses an LLM to condense the chat history and follow-up into a standalone question before retrieval, then generates the answer using retrieved context plus recent history.

8.5 Evaluation

Q40. How do you evaluate a RAG system?

Separately evaluate retrieval (hit rate, recall@k, precision@k, MRR, nDCG on a labelled set) and generation (faithfulness, answer relevancy, context precision and recall, correctness) using tools like RAGAS or DeepEval with LLM-as-a-judge, plus latency, cost and user feedback.

Q41. What is faithfulness?

The share of claims in the answer that can be supported by the retrieved context. Low faithfulness means hallucination even if the answer sounds right.

Q42. Context precision vs context recall?

Context precision: are the retrieved chunks relevant, and are relevant ones ranked at the top (low noise)? Context recall: does the retrieved context contain everything needed for the ground-truth answer (nothing missing)?

Q43. How do you build an evaluation dataset without labels?

Generate synthetic question–answer–context triples from your documents with an LLM (RAGAS testset generator), have humans review a sample, and add real user questions and failures from production over time.

Q44. What are the limitations of LLM-as-a-judge?

Bias (position, verbosity, self-preference), inconsistency, and cost. Mitigate with clear rubrics, low temperature, structured outputs, and calibration against human labels.

8.6 Production & Scenarios

Q45. Your RAG chatbot is hallucinating. How do you debug it?

First check retrieval: log retrieved chunks for failing queries. If the right chunk is missing, fix chunking, use hybrid search, query rewriting or re-ranking. If the right chunk is present but the answer is wrong, tighten the prompt ('only from context, else say I don't know'), lower temperature, reduce noisy context, and add a groundedness check. Measure with faithfulness and context recall before and after.

Q46. Users search with exact error codes and get poor results. Fix?

Dense embeddings handle rare tokens poorly. Add BM25 for hybrid search, store error codes as metadata for filtering, and possibly re-rank.

Q47. Retrieval is good but latency is high. What do you do?

Stream the response; cache embeddings, retrieval results and frequent answers (semantic cache); run retrievers in parallel; reduce k and context size; use a smaller re-ranker or grader model; tune HNSW efSearch; skip retrieval for simple queries (adaptive routing).

Q48. How do you keep the index up to date?

Incremental ingestion: detect new, changed and deleted documents with hashes or timestamps, re-chunk and re-embed only those, upsert by stable IDs, delete stale vectors, and store version and date metadata.

Q49. How do you enforce document-level security?

Store ACLs (user or group IDs) in chunk metadata and apply a metadata filter based on the logged-in user at retrieval time, so unauthorised chunks never reach the LLM.

Q50. How do you protect RAG from prompt injection?

Treat retrieved text as untrusted data: delimit it clearly, instruct the model not to follow instructions inside the context, sanitise ingested content, restrict which sources are indexed, limit tool permissions, and add output guardrails.

Q51. A question needs data from both PDFs and a SQL database. How would you design it?

Use a router or agent: classify the question, send numeric and aggregate parts to a Text-to-SQL tool (read-only), send policy and text parts to the vector retriever, then combine both results in the final generation step. LangGraph works well for this.

Q52. How would you reduce cost of a RAG system?

Use smaller and cheaper models for routing, grading and rewriting; reduce chunk count and size via re-ranking and compression; use prompt caching and semantic caching; batch embeddings; use open-source embeddings.

Q53. What would you log for observability?

User query, rewritten query, retrieved chunk IDs and scores, re-ranked order, final prompt token count, model, answer, latency per step, cost, and user feedback — with tracing via LangSmith or Langfuse.

Q54. Design a RAG system for 10 million internal documents.

Distributed ingestion with layout-aware parsing; structure-aware chunking with rich metadata including ACLs; a scalable vector DB with HNSW or IVF-PQ and sharding; hybrid search plus a cross-encoder re-ranker; query rewriting and routing by department; permission filters; a grounded prompt with citations; caching and streaming; RAGAS-based evaluation in CI; observability and a feedback loop.

8.7 Quick one-liners for rapid-fire rounds

Term	One-liner
Chunk	The unit of text that is embedded, retrieved and fed to the LLM.
Top-k	Number of most similar chunks returned by the retriever.
Embedding	Vector that captures meaning; similar text → nearby vectors.
BM25	Keyword ranking using term frequency, inverse document frequency and length normalisation.
Hybrid search	BM25 + vector search fused (often with RRF).
Re-ranker	Cross-encoder that re-scores candidates for higher precision.
HyDE	Search using the embedding of a hypothetical answer.
MMR	Relevance + diversity selection.
CRAG	Grade retrieved docs; fall back to web search if bad.
Self-RAG	Model reflects: retrieve? relevant? supported? useful?
Agentic RAG	Agent uses retrieval as a tool in a ReAct loop.
GraphRAG	Knowledge graph + community summaries for relational and global questions.
Faithfulness	Is the answer supported by the context?
Lost in the middle	LLMs underuse information placed in the middle of long contexts.

9. Learning Roadmap & Project Ideas

9.1 Prerequisites

- Python, REST APIs, JSON; basic NumPy (vectors, dot product, norms).
- NLP basics: tokenisation, transformers, attention, encoder vs decoder models.
- LLM basics: prompting, temperature, context window, tokens, structured output, streaming.
- Information retrieval basics: TF-IDF, BM25, precision/recall.

9.2 Phased plan

Phase	Learn	Build
1. Foundations	Embeddings, cosine similarity, chunking, vector stores (Chroma/FAISS), prompt templates	Naive RAG over your own PDFs from scratch (Section 5.1)
2. Better retrieval	BM25, hybrid search, RRF, MMR, cross-encoder re-ranking, metadata filters, query rewriting, HyDE, multi-query, parent-document	Hybrid + rerank Q&A bot with citations; compare recall@k before/after
3. Evaluation	Golden datasets, recall@k, MRR, RAGAS metrics, LLM-as-judge, tracing	Evaluation harness that scores every config change
4. Control flow	LangGraph state, conditional edges, loops; CRAG, Self-RAG, Adaptive RAG, conversational memory	CRAG + Self-RAG graph with web fallback and hallucination grader
5. Advanced data	GraphRAG, Text-to-SQL, multimodal RAG, contextual retrieval	Assistant that answers from PDFs + SQL tables + images
6. Production	FastAPI serving, streaming, caching, ACLs, prompt-injection defence, observability, incremental indexing, Docker	Deployed RAG API with Streamlit UI, logging and feedback

9.3 Portfolio project ideas

- **IT incident knowledge assistant**: hybrid search over past tickets and runbooks; error codes via BM25; CRAG fallback.
- **Policy/HR chatbot** with document-level access control and citations.
- **Research assistant**: agentic RAG with arXiv + web search tools and multi-hop reasoning.
- **Customer support RAG** with conversational memory, routing to FAQ vs order database (SQL).
- **Financial report analyst**: contextual retrieval + table handling + GraphRAG for company relationships.

9.4 One-page cheat sheet

Problem	Go-to technique
Vague / short query	Query rewriting, Multi-query, HyDE
Exact terms / IDs missed	Hybrid search (BM25 + dense)
Right doc retrieved but ranked low	Bigger k + cross-encoder rerank
Chunks lack context	Parent-document, sentence-window, contextual retrieval
Irrelevant docs retrieved	Re-ranking, thresholds, CRAG grader
Answer not supported by docs	Strict prompt, Self-RAG hallucination grader, faithfulness eval
Follow-up questions	Conversational RAG (condense question)

Problem	Go-to technique
Needs multiple steps	Query decomposition, multi-hop, agentic RAG
Relationships / global themes	GraphRAG
Numbers from tables	Text-to-SQL / structured RAG
Images and diagrams	Multimodal RAG
Mixed simple and complex traffic	Adaptive RAG (router)
Knowledge base incomplete	CRAG with web-search fallback

Interview tip: Golden rule for interviews and real projects: **measure first, then optimise**. Start with naive RAG, build an evaluation set, find whether retrieval or generation is failing, then add only the technique that fixes that failure.